

Unicode Friendly Encoding Conversions for the Standard Library (R1)

Proposes character encoding conversion and related functions to ease interoperability between strings and other sequences of character types `char`, `char16_t`, `char32_t`, and `wchar_t`. Support for Unicode Transformation Form (UTF) and wide character encodings is built-in, while narrow character encodings are supported via traditional `codecvt` facets. Pure addition to the standard library. No changes to the core language or existing standard library components. Breaks no existing code or ABI. [Proposed wording](#) provided. Specified in accordance with ISO/IEC 10646 and the Unicode Standard. Has been implemented. Suitable for either a library TS or the standard itself.

[Major interface revision](#): R1 adds important functionality yet markedly reduces interface size.

Introduction and Motivation

C++ types `char`, `char16_t`, and `char32_t` support character and string literals encoded in Unicode Transformation Forms UTF-8, UTF-16, and UTF-32 respectively. Additional narrow character encodings are supported by the standard library's `codecvt` facets via conversion to a wide character encoding.

Users may need to use multiple encodings in the same application, or even the same function. Yet neither the language nor the standard library provides a convenient C++ way to convert between these encodings, let alone a way to do that securely. There is no equivalent to the ease with which the `std::to_string` family of functions can convert an arithmetic value to a string.

This proposal markedly eases the problems encountered by users due to the lack of convenient encoding conversion in the standard library. Knowledge of UTF-8, UTF-16, UTF-32, and the implementation defined `wchar_t` wide encoding is built-in to make the interface Unicode friendly. The interface meets the error handling requirements of the Unicode and ISO/IEC 10646 standards, and meets the error handling needs requested by Unicode experts.

Example

Problem: Convert a string `s` to UTF-16 in a way that converts the string types and encodings, and handles errors according to the best practices documented in the ISO/IEC 10646 and Unicode standards.

Using the proposal:

```
to_string<utf16>(s);
```

Where `s` can be anything convertible to `std::basic_string_view<char/char16_t/char32_t/wchar_t>` encoded in the associated UTF-8, UTF-16, UTF-32, or wide character encoding. If `s` has a value type of `char`, but is not UTF-8 encoded, a second argument supplies a `std::codecvt` derived facet that converts to an internal type such as `wchar_t` or `char32_t` and its associated encoding:

```
to_string<utf16>(s, ccvt_facet);
```

Without the proposal, using only the standard library: This might not be too difficult using a third-party library, but is surprisingly difficult using only the standard library. Unless the developer has enough Unicode experience to focus on error detection and to test against an existing test data set, a roll-your-own solution would probably be very time consuming and error-prone.

Prior proposal

[N3398](#), *String Interoperation Library*, proposed a complete overhaul of the standard library's mechanisms for character encoding conversion. The proposal was discussed at the Portland meeting in 2012. Some aspects of the proposal drew strong support, such as improving Unicode string interoperability. Other aspects drew strong opposition, such as new low level functionality to replace `std::codecvt`. Clearly participants did not want N3398 - they wanted a different proposal,

less overreaching and more focused on Unicode encoding conversions. Bill Plauger summed it up when he said something like "Don't reinvent codecvt. That said, we should pick a winner — Unicode."

This P0353 proposal is completely new and not a revision of N3398.

Revision history

R1 - Pre-Issaquah mailing

- Major interface revision: Discussion in Oulu and experimental use of the R0 interface exposed serious issues:
 - Users who need UTF-based encoding conversion also often need codecvt-based encoding conversion and visa versa. Yet R0 did not provided codecvt-based conversion.
 - The R0 interface had 18 signatures spread over three levels of functionality, and that was very confusing. Adding codecvt-based conversion would have doubled the number of signatures further increasing user confusion.

The R1 interface provides both UTF-based and codecvt-based encoding conversion, yet consists of only two levels of functionality and two conversions functions; a recode algorithm with a single signature and a `to_string` convenience function with four overloads.

- Added [UTF encoding query](#) functions.
- Added [Requirements](#) section.
- Assume the C++ WP will be changed to specify ISO/IEC 10646:2014, and so that will be the base document for references.
- Lots of general refinement of the proposed wording.

Implementation

A Boost licensed preliminary implementation is available at github.com/beman/unicode.

Acknowledgements

Jeffrey Yasskin, Titus Winters, Michael Spenser, and Fabio participated in a small group discussion of P0353R0 in Oulu. Lots of direction and specific comments that came out of the discussion are reflected in P0353R1. For example, removal of the requirement that `wchar_t` be UTF encoded and need to be more explicit about narrow encoding.

Tom Honermann provided insights about environments no based on POSIX or Windows, such as IBM's z/OS.

Alisdair Meredith, Eric Niebler, Howard Hinnant, Jeffrey Yasskin, Marshall Clow, PJ Plauger, and Stephan T. Lavavej participated in the Portland discussion of N3398. Many of the design decisions that have gone into the current proposal flow directly from the Portland discussion.

After the Portland meeting, Matt Austern sat down with Google's "Unicode people" to "clarify things". His summary of that discussion was very helpful. Its guidance on error handling is reflected in current proposal.

Design decisions

Limit this proposal to encoding conversion and other encoding-related functionality

- Getting UTF encoding conversion right is hard enough without also coping with other Unicode needs.
- No all-encompassing Unicode proposal is on the horizon.
- Other needs requested by domain experts, such as code point iterators, are important enough to deserve their own proposals.
- Guidance from both the committee and domain experts clearly favors a proposal focusing on UTF conversions.

Build in support for UTF-8, UTF16, UTF-32 and wide (i.e. `wchar_t`) encodings

- The UTF encodings are a critical need for the standard to become Unicode friendlier.
- Support for the implementation defined wide character encoding ensures support for much existing code.

Build in support for existing narrow to/from wide codecvt facets

- Allows this brand-new library to immediately support non-UTF encodings without having to repeat two decades of codecvt facet development.

Provide two levels of functionality

The `recode` function provides a recoding conversion algorithm. It operates on an input sequence and produces an output sequence, so provides STL-like functionality to meet generic needs.

The `to_string` function templates provide convenient encoding conversion for `string_view`, `u16string_view`, `u32string_view`, and `wstring_view` arguments, and are intended to complement the existing standard library `to_string` family of functions.

Provide a coherent error detection and handling policy

- Follow the Unicode standard's requirement that "A conformant encoding form conversion will treat any ill-formed code unit sequence as an error condition." (Unicode 3.9 D93 and C10).
- Provide a note to inform users of as to when they do and do not have to explicitly check for errors themselves.
- Provide error checking functions so that users can also explicitly check for errors to meet application needs.
- Handle errors via function objects to support the varied user needs described by domain experts.
- Default error handler function object follows the Unicode standard's recommendation of U+FFFD as a replacement character. Their rationale is that the other common approaches, including throwing an exception, can be and have been used as attack vectors.

Provide encoding conversions as explicitly called non-member functions

- Avoids possibly expensive hidden automatic encoding conversions in unexpected places.
- Avoids the need to change existing standard library components.
- Builds on the other STL algorithms and the `to_string` functions already in the standard library.

Place the proposed components in namespace `unicode`

- Emphasizes that these functions assume char strings use a Unicode encoding.
- Signals that the committee cares about Unicode, but doesn't want to force it on users who prefer other encodings.
- Provides a home for these and future Unicode specific functions.

Keep interfaces neutral as to which character type or UTF encoding is "best"

- Each of these encodings have uses where it is preferred or required, and all of these needs may appear in the same application. For example:
 - UTF-8 is required by the API's for some operating systems.
 - UTF-16 is required to interfacing with existing databases that use UTF-16 encoding.
 - UTF-32 is preferred in code where every Unicode code point being encoded as a single code unit is advantageous.

Provide types `narrow`, `utf8`, `utf16`, `utf32`, and `wide` to identify encodings

- String `value_type` alone is insufficient because in the case of char it is ambiguous.
- The resulting user code more explicit and so easier to read.

Base conformance and definitions on ISO/IEC 10646:2014

- Rewriting specifications that are already covered by another standard, and then trying to keep that wording in sync as the other standard evolves, is a path to insanity.

Use variadic templates to minimize interface surface area

- Converting between each two pairs of five encodings (`narrow`, UTF-8, UTF-16, UTF-32, `wide`) results in twenty-five possible conversion functions. Providing conversion in both algorithm form and convenience function form doubles the number of functions to fifty. Initially, use of templates reduced the number of functions quite a bit, but trial use of an implementation for real work was discouraging; it was too easy to get the arguments right but then call the wrong function name.
- Use of variadic templates (i.e. a generative approach) reduced the interface to two functions - one `recode` conversion algorithm and one `to_string` convenience function. These closely match the abstraction and mental model of their behavior; the differing arguments are just a detail. Static asserts can ensure comprehensible error messages when contradictory argument types are passed.

To Do

- Use case (which would also make a good example): when recoding "Krügler" to ASCII, output "Kruegler". See en.wikipedia.org/wiki/Ü. Approach: Loop over `find_first_ill_formed()`. Approach rejected as overly complicated: Revise error handling to (1) add arguments specifying range of ill-formed input, (2) return a `basic_string<ToCharT>`. The concern has more to do with added implementation complexity than interface complexity, so consider if `imp` could call (in an inline detail function) `first_ill_formed()` to get the ill-formed range. Would require `ForwardIterators`, which is probably OK.
- Add an optional `codecvt` argument to `first_ill_formed()` so it can also be applied to narrow strings.

Proposed wording

*This wording assumes P0417 C++17 should refer to ISO/IEC 10646 2014 instead of 1994 **has been accepted into the C++ working paper.***

Unicode library [ucs]

This sub-clause describes components that C++ programs may use to perform operations on characters, strings, and other sequences of characters encoded in various encoding forms. Encoding forms UTF-8, UTF-16, and UTF-32 are supported, as are narrow character encodings having a `codecvt` facet meeting requirements described below.

[*Note:* The C++ standard does not require the encoding of `char`, `char16_t`, `char32_t`, and `wchar_t` strings be UTF encoded, although `u8`, `u`, and `U` string literals are UTF encoded. The components in this sub-clause use the provided types `narrow`, `utf8`, `utf16`, `utf32`, and `wide` to identify specific encodings [uni.encoding]. — *end note*]

Normative references [ucs.refs]

Within this sub-clause a reference written in the form "(UCS *number*)" refers to section *number* of ISO/IEC 10646:2014 (C++ [intro.refs]).

[*Note:* ISO/IEC 10646 Universal Coded Character Set (UCS) is the ISO/IEC standard for [Unicode](#). It is synchronized with *The Unicode Standard* maintained by the Unicode Consortium. — *end note*]

[Footnote] Unicode® is a registered trademark of Unicode, Inc. This information is given for the convenience of users of this document and does not constitute an endorsement by ISO or IEC of this product.

Definitions [ucs.defs]

The definitions from (UCS 4.) apply throughout. [*Examples:* code point (UCS 4.10), code unit (UCS 4.11), encoding form (UCS 4.23), ill-formed code unit sequence (UCS 4.33), minimal well-formed code unit sequence (UCS 4.41), well-formed code unit sequence (UCS 4.61). — *end examples*]

Encoded character types [ucs.defs.enc-char-type]

The types `char`, `char16_t`, `char32_t`, and `wchar_t`.

Minimal code unit sequence [ucs.def.min-cus]

- If present, a minimal well-formed code unit sequence ([ucs.def.min-well-cus]), otherwise
- a minimal ill-formed code unit sequence ([ucs.def.min-ill-cus]).

Minimal ill-formed code unit sequence [ucs.def.min-ill-cus]

- If present, the longest initial subset of a minimal well-formed code unit sequence, followed by
- all subsequent code units not valid as the initial code unit of a minimal well-formed code unit sequence.

Minimal well-formed code unit sequence [ucs.def.min-well-cus]

Determined by the encoding type [uni.encoding]:

- For encoding types `utf8`, `utf16`, and `utf32`, a well-formed code unit sequence (UCS 4.61) that maps to a single UCS scalar value.
- For encoding type `narrow`, implementation defined according to a facet argument of a type derived from `std::codecvt`.
- For encoding type `wide`, implementation defined.

Requirements [ucs.req]

Template parameters named `InputIterator` shall satisfy the requirements of an input iterator (C++ [input.iterators]).

Template parameters named `ForwardIterator` shall satisfy the requirements of an forward iterator (C++ [forward.iterators]).

Template parameters named `OutputIterator` shall satisfy the requirements of an output iterator (C++ [output.iterators]).

Header `<experimental/unicode>` synopsis

```
namespace std {
namespace experimental {
inline namespace fundamentals_v2 {
namespace unicode {

    // [uni.encoding] encoding types
    struct narrow {using value_type = char;}; // codecvt determined encoding
    struct utf8 {using value_type = char;}; // UTF-8 encoding
    struct utf16 {using value_type = char16_t;}; // UTF-16 encoding
    struct utf32 {using value_type = char32_t;}; // UTF-32 encoding
    struct wide {using value_type = wchar_t;}; // wide-character literal
    // encoding [lex.ccon]

    // [uni.is_encoding] is_encoding type-trait
    template <class T> struct is_encoding : public false_type {};
    template<> struct is_encoding<narrow> : true_type {};
    template<> struct is_encoding<utf8> : true_type {};
    template<> struct is_encoding<utf16> : true_type {};
    template<> struct is_encoding<utf32> : true_type {};
    template<> struct is_encoding<wide> : true_type {};

    template <class T> constexpr bool is_encoding_v = is_encoding<T>::value;

    // [uni.is_encoded_character] is_encoded_character type-trait
    template <class T> struct is_encoded_character : public false_type {};
    template<> struct is_encoded_character<char> : true_type {};
    template<> struct is_encoded_character<char16_t> : true_type {};
    template<> struct is_encoded_character<char32_t> : true_type {};
    template<> struct is_encoded_character<wchar_t> : true_type {};

    template <class T> constexpr bool is_encoded_character_v
        = is_encoded_character<T>::value;

    // [uni.err] default error handler
    template <class CharT> struct ufffd;
    template <> struct ufffd<char>;
    template <> struct ufffd<char16_t>;
    template <> struct ufffd<char32_t>;
    template <> struct ufffd<wchar_t>;

    // [uni.recode] encoding conversion algorithm
    template <class FromEncoding, class ToEncoding,
              class InputIterator, class OutputIterator, class ... T>
    OutputIterator recode(InputIterator first, InputIterator last,
        OutputIterator result, const T& ... args);

    // [uni.to_string] string encoding conversion
    template <class ToEncoding = utf8, class ...Pack>
    basic_string<typename ToEncoding::value_type>
    to_string(string_view v, const Pack& ... args);
    template <class ToEncoding = utf8, class ...Pack>
    basic_string<typename ToEncoding::value_type>
    to_string(u16string_view v, const Pack& ... args);
    template <class ToEncoding = utf8, class ...Pack>
    basic_string<typename ToEncoding::value_type>
    to_string(u32string_view v, const Pack& ... args);
    template <class ToEncoding = utf8, class ...Pack>
    basic_string<typename ToEncoding::value_type>
    to_string(wstring_view v, const Pack& ... args);

    // [uni.utf-query] Encoding queries
    template <class ForwardIterator>
    std::pair<ForwardIterator, ForwardIterator>
    first_ill_formed(ForwardIterator first, ForwardIterator last) noexcept;
    bool is_well_formed(string_view v) noexcept;
    bool is_well_formed(u16string_view v) noexcept;
    bool is_well_formed(u32string_view v) noexcept;
    bool is_well_formed(wstring_view v) noexcept;
```

```
} // namespace unicode
} // namespace fundamentals_v2
} // namespace experimental
} // namespace std
```

Character type, encoding type, and encoding relationships [uni.encoding]

The types narrow, utf8, utf16, utf32, and wide provided by header <unicode> identify the encoding of strings and sequences of the encoded character types ([ucs.defs.enc-char-type]).

[Note: Users must supply arguments of std::codecvt derived types for operations on narrow encoded strings and sequences. For the other encoding types, such facets are not necessary. — end note]

The relationship between encoded character types, encoding types, and encodings is specified by the following table:

Table of Relationships		
Character type	Encoding type	Encoding
char	narrow	The encoding of characters of type char for a user-supplied type derived from std::codecvt<Elem, char, std::mbstate_t> where Elem is wchar_t or char32_t. Implementations are permitted to support additional implementation defined types. The derived type shall meet the requirements of the standard code-conversion facet std::codecvt<Elem, char, std::mbstate_t>. (C++ [locale.codecvt]).
	utf8	UTF-8 (UCS 9.2).
char16_t	utf16	UTF-16 (UCS 9.3).
char32_t	utf32	UTF-32 (UCS 9.4)
wchar_t	wide	The implementation defined encoding of wide-character literals (C++ [lex.ccon]).

Error handling [uni.err]

When an ill-formed code unit subsequence is detected during execution of a conversion function, an *error handler* function object shall be invoked. Unless the error handler throws an exception, the string returned by the error handler shall be added to the output sequence and the ill-formed input code unit subsequence shall not be converted and added to the output sequence. Detection and error handling for ill-formed code unit subsequences is required even when the input and output encodings are the same. [Note: If the error handler function object always returns a pointer to a well-formed code point sequence, the conversion function's entire output sequence will be a well-formed code point sequence. — end note]

```
template <class CharT> struct ufffd;
template <> struct ufffd<char>;
template <> struct ufffd<char16_t>;
template <> struct ufffd<char32_t>;
template <> struct ufffd<wchar_t>;
```

struct ufffd is the default error handler function object for conversion functions. The default error handling function object returns U+FFFD REPLACEMENT CHARACTER as a single code point error marker. Each specialization shall provide a member function with the signature:

constexpr const CharT* operator()() const noexcept;

that returns a pointer to the value indicated in the Specializations table:

Specializations	
CharT	Returns
char	u8"\uFFFD"
char16_t	u"\uFFFD"
char32_t	U"\uFFFD"
wchar_t	L"\uFFFD"

[Note: U+FFFD REPLACEMENT CHARACTER is returned as the default single code point error marker in accordance with the recommendations of the Unicode Standard. The rationale given by the Unicode standard is essentially that other commonly used approaches, including throwing exceptions, can be and have been used as security attack vectors. —end note]

Encoding conversion algorithm [uni.recode]

```
template <class FromEncoding, class ToEncoding,
```

```

class InputIterator, class OutputIterator, class ... T>
OutputIterator recode(InputIterator first, InputIterator last,
    OutputIterator result, const T& ... args);

```

Effects: For each minimal code unit subsequence in the range [first, last):

- If the code unit subsequence is well-formed, convert the code point it represents from the input sequence encoding to the output sequence encoding and then copy the code units of that code point as if by `*result++ = *u++` where `u` is an iterator over the code units making up the code point.
- Otherwise, copy the null-terminated string returned by the `eh` function object to the output result as if by `*result++ = *p++` where `p` iterates over the string returned by `eh`.

Returns: `result`.

Remarks: An implementation is permitted to first convert from the input encoding to an intermediate encoding, and then convert the intermediate encoding to the output encoding. [Note: This allows implementations to perform conversions to or from narrow via an intermediate string of a *Codecvt* argument's *intern_type* and encoding. —end note]

The requirements for the `args` parameter pack arguments are shown in the following table.

<i>Parameter pack argument requirements</i>				
FromEncoding	ToEncoding	First args argument	Second args argument	Third args argument
utf8, utf16, utf32, or wide	utf8, utf16, utf32, or wide	Optional error handler function object [uni.err]	Not allowed; diagnostic required	Not allowed; diagnostic required
utf8, utf16, utf32, or wide	narrow	const <i>Codecvt</i> &	Optional error handler function object [uni.err]	Not allowed; diagnostic required
narrow	utf8, utf16, utf32, or wide	const <i>Codecvt</i> &	Optional error handler function object [uni.err]	Not allowed; diagnostic required
narrow	narrow	const <i>Codecvt</i> &	const <i>Codecvt</i> &	Optional error handler function object [uni.err]

Type *Codecvt* is the `std::codecvt` derived type described in the [uni.encoding] table. Used to perform the conversion to `char` from `Elem`.

Postcondition: If the string returned by each call to the `eh` function object during the execution of the algorithm is a well-formed code point sequence, then the output sequence is a well-formed code point sequence.

String encoding conversion [uni.to_string]

```

template <class ToEncoding, class ...Pack>
basic_string<typename ToEncoding::value_type>
to_string(string_view v, const Pack& ... args);
template <class ToEncoding, class ...Pack>
basic_string<typename ToEncoding::value_type>
to_string(u16string_view v, const Pack& ... args);
template <class ToEncoding, class ...Pack>
basic_string<typename ToEncoding::value_type>
to_string(u32string_view v, const Pack& ... args);
template <class ToEncoding, class ...Pack>
basic_string<typename ToEncoding::value_type>
to_string(wstring_view v, const Pack& ... args);

```

Effects: Equivalent to:

```

basic_string<typename ToEncoding::value_type> tmp;
recode(<FromEncoding, ToEncoding>(v.cbegin(), v.cend(),
    back_inserter(tmp), args ...));
return tmp;

```

For the first overload, *FromEncoding* is narrow if there are two function arguments convertible to `ccvt_type`, and *FromEncoding* is narrow and if there is one argument convertible to `ccvt_type` and *ToEncoding* is not

narrow. Otherwise **FromEncoding** is utf8.

For the second, third, and fourth overloads, **FromEncoding** is utf16, utf32, and wide, respectively.

[Example:

```
#include <string_encoding>
#include <string>
#include <locale>
#include <cvt/big5> // vendor supplied
#include <cvt/sjis> // vendor supplied

using namespace std::unicode;
using namespace std;

string sjisstr() { string s; /*load s*/ return s; }
string big5str() { string s; /*load s*/ return s; }

int main()
{
    string  locstr("abc123..."); // narrow encoding known to std::locale()
    string  u8str(u8"abc123$€¥..."); // UTF-8 encoded
    u16string u16str(u"abc123$€¥..."); // UTF-16 encoded
    u32string u32str(U"abc123$€¥..."); // UTF-32 encoding
    wstring WSTR(L"abc123$€¥..."); // implementation defined wide encoding

    stdext::cvt::codecvt_big5<wchar_t> big5; // vendor supplied Big-5 facet
    stdext::cvt::codecvt_sjis<wchar_t> sjis; // vendor supplied Shift-JIS facet

    auto loc = std::locale();
    auto& loc_ccvt(std::use_facet<ccvt_type>(loc));

    u16string s1 = to_string<utf16>(u8str); // UTF-16 from UTF-8
    wstring  s2 = to_string<wide>(locstr, loc_ccvt); // wide from narrow
    u32string s3 = to_string<utf32>(sjisstr(), sjis); // UTF-32 from Shift-JIS
    string  s4 = to_string<narrow>(u32str, big5); // Big-5 from UTF-32
    string  s5 = to_string<narrow>(big5str(), big5, sjis); // Shift-JIS from Big-5

    string s6 = to_string<utf8>(u8str); // replace errors with u8"\uFFFD"
    string s7 = to_string(u16str, []() {return "?";}); // replace errors with '?'
    string s8 = to_string(WSTR, []() {throw "barf"; return "";}); // throw on error

    string s9 = to_string<narrow>(u16str, big5); // OK
    string s10 = to_string<utf8>(u16str, big5); // error: ccvt_type arg not allowed
    string s11 = to_string<narrow>(u16str); // error: ccvt_type arg required
    string s12 = to_string<narrow>(u16str, big5, big5); // error: >1 ccvt_type arg
    wstring s13 = to_string<wide>(locstr, big5, big5); // error: >1 ccvt_type arg
    string s14 = to_string<narrow>(locstr); // error: ccvt_type arg required
}
```

— end example]

Encoding queries [uni.encoding-query]

These functions determine whether or not character sequences or string views consist of well-formed code unit sequences (UCS 4.61).

```
template <class ForwardIterator>
std::pair<ForwardIterator, ForwardIterator>
first_ill_formed(ForwardIterator first, ForwardIterator last) noexcept;
```

Effects: Equivalent to:

Searches for the first minimal ill-formed code unit subsequence ([ucs.def.min-ill-cus]) in the half-open range [first, last).

If such a minimal ill-formed code unit subsequence is found, returns std::make_pair(begin, end) where begin is an iterator to the first element of the minimal ill-formed code unit subsequence and end is a past-the-end iterator for the past-the-end element of the minimal ill-formed code unit subsequence.

Otherwise returns std::make_pair(last, last).

Returns: See *Effects*.

Remarks: The specific encoding form is determined by the ForwardIterator value type ([uni.encoding]).

```
bool is_well_formed(string_view v) noexcept;
```



```
bool is_well_formed(u16string_view v) noexcept;  
bool is_well_formed(u32string_view v) noexcept;  
bool is_well_formed(wstring_view v) noexcept;
```

Returns: Equivalent to `first_ill_formed(v.cbegin(), v.cend()).first == v.cend()`.
